PROGRAMA DE ESTUDIANTES

Diseño del problema

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

UNIVERSIDAD

8 DE FEBRERO DE 2026

JUAN SEBASTIAN GARAVITO MEJIA

Descripción del problema

El programa le solicita al usuario que ingrese los nombres y notas de los estudiantes. A partir de las notas ingresadas mediante dividir y conquistar se suman las notas y se calcula el promedio general para después determinar cuáles estudiantes son los estudiantes que aprobaron.

Historial de usuario

H01 - Ingresar nombre y notas de estudiantes

- **Descripción**
  El sistema le solicita al usuario el nombre y las notas.
- **Caso exitoso**
  Se ingresa nombres y notas en orden correcto sin puntos o comas entre nombre y nota.
- **Caso no exitoso**
  El usuario ingresa los datos en orden diferente y/o con puntos o comas entre datos.
- **Entrada**
  Nombres y notas.
  Ejemplo: juan 4.2 diana 3.0 maría 4.9
- **Salida**
  No hay.

H02 – Evaluar estudiantes aprobados

- **Descripción**
  El sistema evalúa los estudiantes que cumplen la condición para aprobar.
- **Caso exitoso**
  Se muestra el nombre de los estudiantes aprobados.
- **Entrada**
  Lista nombres y notas.
- **Salida**
  Nombres de los estudiantes que aprobaron.

Requerimientos funcionales

RF01 – Solicitar nombres y notas de los estudiantes.

RF02 – Almacenar los nombres y notas de los estudiantes.

RF03 – Calcular la suma de las notas con conquistar y dividir.

RF03 – Calcular el promedio de las notas.

RF04 – Comparar las notas con el promedio.

RF05 – Mostrar los nombres de los estudiantes que aprobaron.

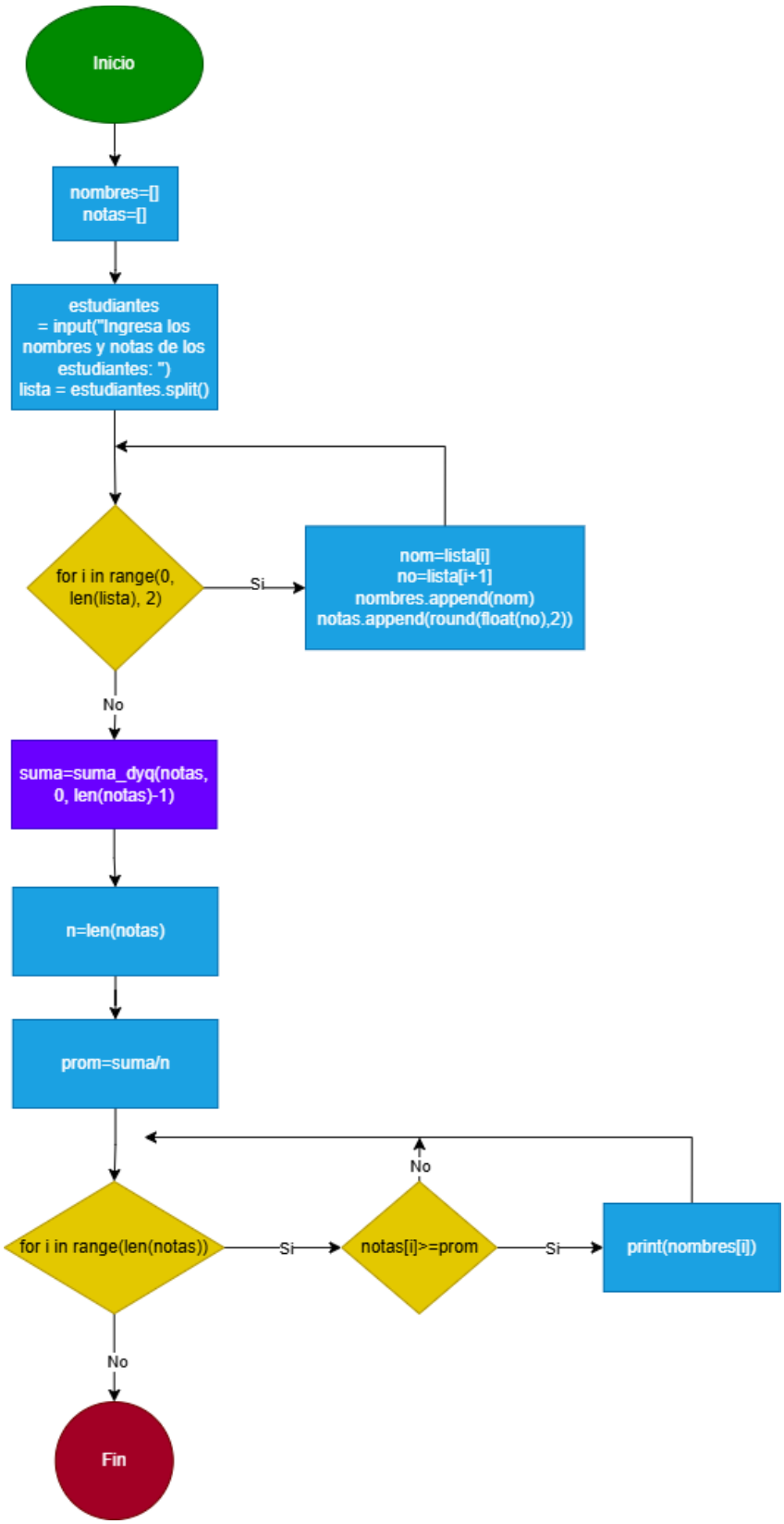
Requerimientos no funcionales

- Debe ser desarrollado en Python.

- Dividir y conquistar debe ser implementado para calcular la suma de las notas.

Diagrama de flujo:

Main



D&Q

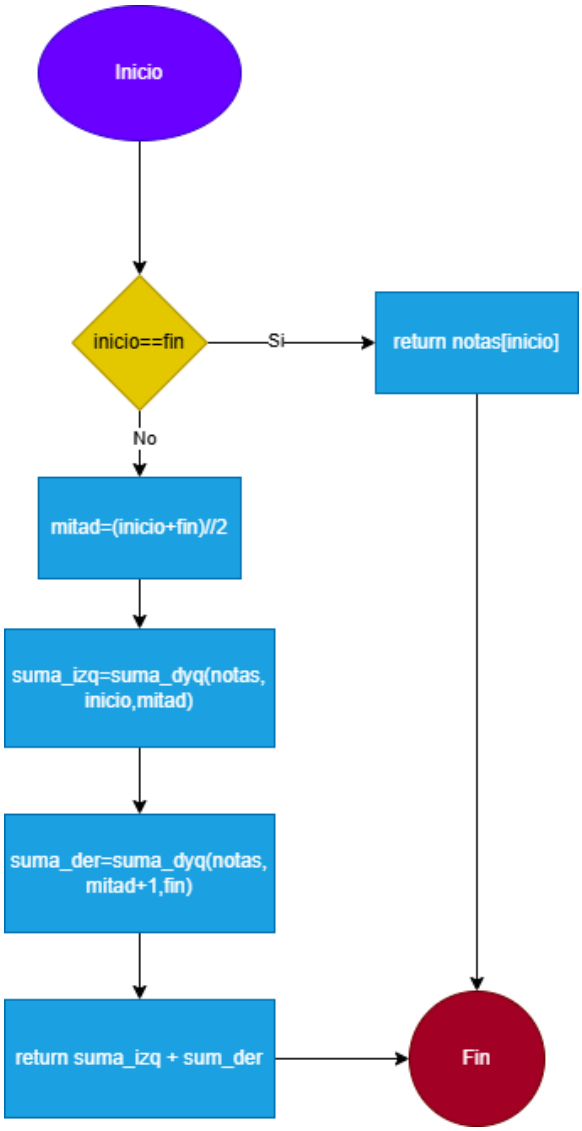


Diagrama casos de uso

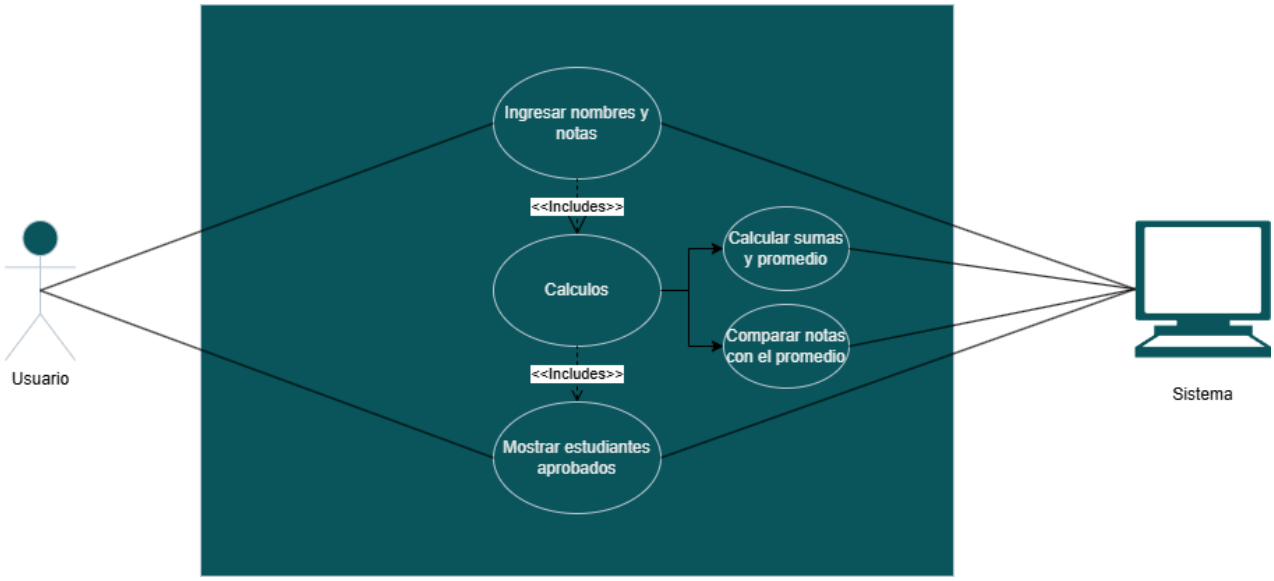
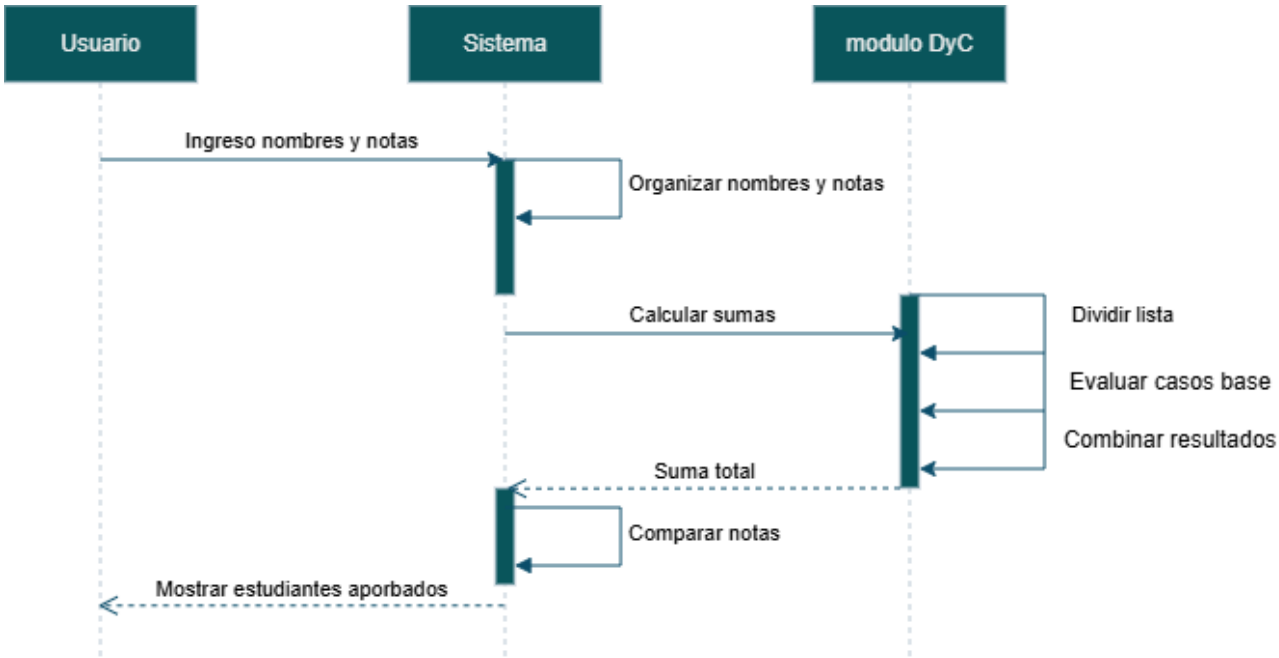


Diagrama de secuencia



Análisis de complejidad

- **Costo tiempo**

Línea del programa (suma_divide_conquista)	**Costo**	**Tiempo**
if inicio == fin:	c_1	n
return notas[inicio]	c_2	$1; inicio == fin$
mitad = (inicio + fin) // 2	c_3	$(n-1)$
suma_izq = suma_divide_conquista(notas, inicio, mitad)	$c_5 + T(\frac{n}{2})$	1
suma_der = suma_divide_conquista(notas, mitad + 1, fin)	$c_6 + T(\frac{n}{2})$	1
return suma_izq + suma_der	c_6	$(n-1)$

T(n):

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Donde:

$$a = 2; b = 2; f(n) = O(n)$$

Reemplazamos

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$\underline{T(n) = O(n)}$$

Línea del programa (main)	**Costo**	**Tiempo**
nombres = []	c_1	1
notas = []	c_2	1
estudiantes = input("Ingresa los nombres y notas de los estudiantes: ")	c_3	1
lista = estudiantes.split()	c_4	1
for i in range(0, len(lista), 2):	c_5	$(\frac{n}{2}+1)$
nom = lista[i]	c_6	$\frac{n}{2}$
no = lista[i + 1]	c_7	$\frac{n}{2}$
nombres.append(nom)	c_8	$\frac{n}{2}$
notas.append(round(float(no), 2))	c_9	$\frac{n}{2}$
suma = suma_divide_conquista(notas, 0, len(notas) - 1)	c_{10}	$\Theta(n)$
n = len(notas)	c_{11}	1
prom = suma / n	c_{12}	1
for i in range(len(notas)):	c_{13}	n
if notas[i] >= prom:	c_{14}	$(n-1); notas[i] \geq prom$
print(nombres[i])	c_{15}	$(n-1); notas[i] \geq prom$

T(n):

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Donde:

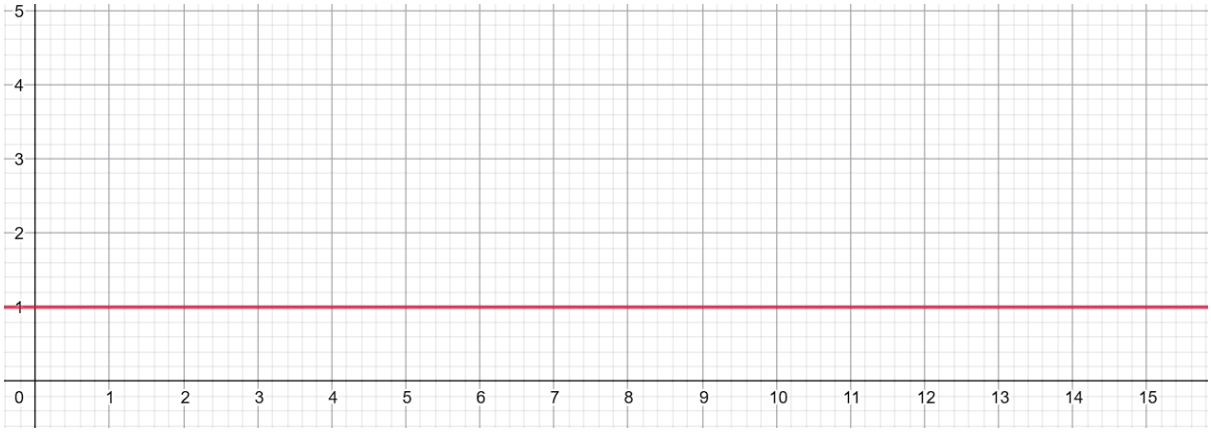
$$a = 2; b = 2; f(n) = O(n)$$

Reemplazamos

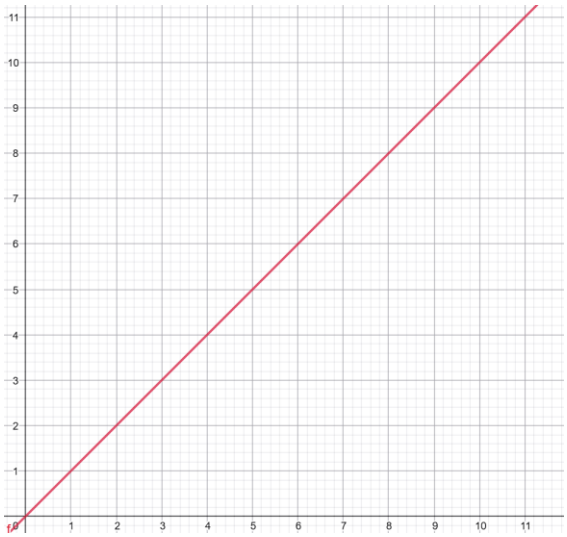
$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = O(n)$$

- **Big oh:** $O(n)$
- **Mejor caso:**
  Constante
  Grafica



- **Peor caso:**
  Recta
  Grafica



- **Análisis propio**
  El programa desarrollado solicita el nombre y las notas de varios estudiantes, calculando el promedio y determinando los estudiantes cuya nota es igual o mayor al promedio calculando (estudiantes aprobados). Para ello, el algoritmo realiza la suma de las notas mediante dividir y conquistar, esto con el fin de dividir el problema en subproblemas y llegar a ser mas eficientes. Seguidamente, se combinan los subproblemas y se calcula el promedio de las notas para determinar los estudiantes que aprueban.
  A pesar del uso de recursión, la complejidad del algoritmo sigue siendo lineal O(n), lo que lo convierte en un programa eficiente para manejar un numero variable de estudiantes.